Islamic University of Technology (IUT)

Department of Electrical and Electronic Engineering

Complex Engineering Problem Report

Design and Implementation of a Low-Cost
Handheld Tic-Tac-Toe Gaming Device
on the AT89C51 Microcontroller Platform

Course No.	EEE 4705
Course Name	Microcontroller Based System Design
Academic Year / Semester	2024-25 / Winter
Course Instructor	Prof. Nadim Ahmed
Project Team	RetroLogic Embedded Systems Design Team
Submission Date	April 17, 2026

Group: C2
Team no: 1

Team members:

Student name	Student ID
Ar-Rafi Ishraq	210021330
Md. Iftimum Ul Haque	210021326
Miah Rashnan Wahid Purna	210021334
Sharia Hamza	210021354
Fakri Issihaka	210021352

1. Introduction

1.1 Problem Statement

RetroLogic, an emerging startup in the handheld-electronics segment, aims to capitalize on the growing nostalgia-driven toy and game market by launching a standalone handheld Tic-Tac-Toe gaming device. The device must be sold at a price point that is attractive to the mass consumer market, operate reliably from portable batteries, and most importantly to deliver a polished, intuitive user experience despite being built entirely on the legacy 8051 microcontroller architecture. The project's core engineering challenge is therefore to strike a workable balance between a rich game-play experience and extremely tight resource constraints.

The design team was asked to propose, build, and validate a complete embedded solution, hardware schematic, software implementation in pure 8051 Assembly language, simulation evidence, and a production-ready costing, suitable for a commercial handheld gaming device.

1.2 Design Objectives

- Implementing a complete 3×3 Tic-Tac-Toe game that runs on the AT89C51 microcontroller using 8051 Assembly Language, with no dependence on C libraries or high-level runtime components.
- Supporting two distinct gameplay modes: Player-vs-Player (PvP) and Player-vs-AI (PvAI), where the AI is implemented as a deterministic rule-based engine.
- Providing clear visual feedback for every game state, turn indication, move placement, invalid moves, win, loss, and draw.
- Keeping the bill-of-materials (BOM) cost low enough to support mass-market pricing in Bangladesh (target < ৳1500 prototype cost).
- Designing for battery operation with reasonable autonomy (≥ 20 hours on commonly available AA cells).
- Respecting the resource budget of the AT89C51: 4 KB of Flash program memory, 128 bytes of internal RAM, and 32 general-purpose I/O pins spread across four ports.

1.3 Summary of System Constraints

Constraint Dimension	Imposed Limit / Target
Platform	Must be an 8051-family MCU (AT89C51 chosen); no high-level compiled code.
Program memory	4 KB of on-chip Flash; game engine, AI, and display driver must fit.
Data memory	128 bytes internal RAM for board state, turn flags, and stack.
I/O resources	32 pins across P0 - P3; P0 also acts as the data bus for the LCDs.
Production cost	Keeping parts cost well below the retail target so that the margin, casing, packaging and labour can all be accommodated within a mass-market price.
Power consumption	Must be low enough that 3 × AA alkaline cells deliver a full school-day of play (≈ 6–8 hours) with comfortable margin.
Usability	No manual required, a child should be able to pick up the device, select a mode, and play within a few seconds.

2. System Architecture

The device is organised around a single AT89C51 microcontroller operating as the central computation and control element. External to the MCU sit three functional blocks: a push-button input matrix, a dual-LCD output subsystem, and the supporting analogue circuitry (crystal oscillator, reset network, and battery-derived 5 V supply). A high-level block diagram of the complete system is shown in Figure 1.

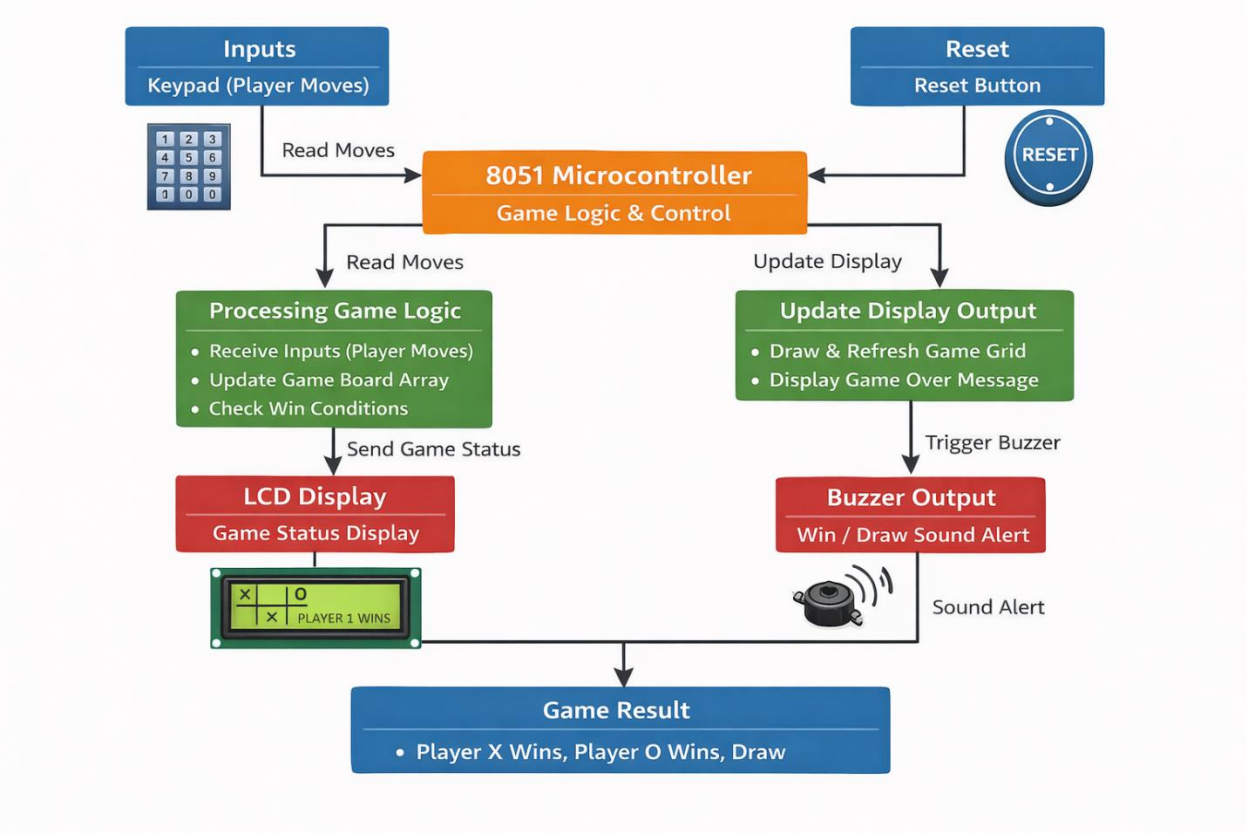


Figure 1: High-level block diagram of the handheld Tic-Tac-Toe device.

2.1 Architectural Rationale

Placing all intelligence inside a single 8-bit microcontroller keeps the Bill of Materials short and the PCB small, both of which are critical for a low-cost handheld product. The push-button interface is connected through Port P2 (and a few spare P3 pins), which gives the firmware simple polling-driven scanning without requiring any external decoder ICs. The two 20×4 alphanumeric LCD modules share Port P0 as an 8-bit data bus and each have their own independent E (enable) control line on Port P3, this 'Shared-bus, split-enable' arrangement lets one Port drive both displays while still treating them as two separate logical screens.

2.2 Resource Budget and Allocation

The allocation of the AT89C51's limited resources to the various subsystems is summarised in the table below. Even after accounting for the display driver, game engine and AI routines, the final compiled program occupies only a fraction of the available flash, leaving comfortable headroom for future feature extensions (sound effects, score persistence, difficulty levels).

Resource	Used / Total	Allocation Notes
Flash program memory	~1.3 KB / 4 KB	Approximately 32% used for Tic-Tac-Toe logic, AI decision-making, and LCD drivers.
Internal RAM	~17 B / 128 B	Board state (9B), Game vars (7B: Mode, Turn, Winner, etc.), Stack starts at 70H.
Port P0 (8 bits)	3 / 8	LCD Control lines: RS (P0.0), RW (P0.1), and E (P0.2).
Port P1 (8 bits)	8 / 8	Dedicated 8-bit data bus for the LCD module.
Port P2 (8 bits)	8 / 8	Direct inputs for Keys 1-6, PvP (A), and PvAI (B).
Port P3 (8 bits)	8 / 8	Direct inputs for Keys 7-9, Reset (C), and expansion keys (*, 0, #, D).
Clock source	11.0592 MHz	Standard quartz crystal for precise timing and potential UART expansion.

3. Input and Output Unit Selection

Selection of the input and output components has the single largest influence on the user experience of a handheld gaming device, and simultaneously the largest influence on its BOM cost. The team considered a broad range of alternatives for both domains before settling on the final choice.

3.1 Input Unit Alternatives

A comparative analysis of the four most realistic input options is presented below.

Option	I/O Pins	Unit Cost (BDT)	Usability	Notes
11 individual push buttons (chosen)	11	~60	Excellent	One button per grid cell + 2 control buttons; direct mapping, no mental model required.
4×4 matrix keypad	8	~150	Good	Fewer pins, but players must find grid cells on a lettered keypad; poor ergonomics for children.
Joystick + confirm button	3 (ADC required)	~250	Moderate	AT89C51 has no on-chip ADC; would require an external ADC, inflating cost and complexity.
Capacitive touch keys	9+	~400+	Moderate	Elegant but expensive, requires additional touch-controller IC, drains more power.

3.1.1 Justification for Push Buttons

- Direct mental mapping: One button physically below each grid cell is the most intuitive scheme possible and requires no instructions.
- Cheapest per-unit input component available in local markets (≈ ৳5–6 per tactile switch), keeping the total input cost around ৳60.
- Zero quiescent current draw: Push buttons consume power only when a finger is actually depressing them, maximising battery life.
- Robust and child-friendly: No delicate membranes or sensitive touch surfaces that can fail after thousands of play cycles.
- The AT89C51 has more than enough pins (Port P2 + two P3 pins) to accommodate 11 individual buttons without a matrix, eliminating debounce ambiguity from two-button rollover.

3.2 Output Unit Alternatives

Option	I/O Pins	Unit Cost (BDT)	Visual Appeal	Notes
2 × 20×4 alphanumeric LCD (chosen)	8 data + 4 ctrl	~690 (2 × 345)	Good	Can show the grid AND status text simultaneously; no manual refresh required once written.
Single 16×2 LCD	6	~180	Poor	Too cramped to render a 3×3 grid plus status text.
8×8 LED matrix	16 or SPI	~200	Moderate	Low resolution, X/O look like blobs, needs constant refresh code that would dominate firmware.
Graphical LCD (128×64)	8 data + 6 ctrl	~750	Excellent	Overkill for this game, similar cost, and the firmware would need bitmap rendering routines that exceed the 4 KB flash.
7-segment displays	Multiple	~300	Very Poor	Cannot render the letters 'X' and 'O' adequately and cannot display a 3×3 grid.

3.2.1 Justification for Dual 20×4 Character LCDs

A single 20×4 display is just barely large enough to render a 3×3 board once the familiar '+' separators are drawn between cells, but it leaves no room for persistent status text (current mode, whose turn, scoreboard, error messages). Using two modules; one dedicated to the board and one dedicated to the status area gives a dramatically cleaner user experience at minimal additional cost, because the second LCD shares the same data bus (Port P0) and only needs its own independent enable line. The advantages are:

- The game grid stays on screen at all times and is never erased to show a message.
- Persistent, always-visible status (mode, current player, score, last-move validity) reduces cognitive load for the player.
- Character-LCD controllers (HD44780) use very little current (~2 mA per module) compared with graphical LCD backplanes.
- No dynamic refresh required, write once to the controller's DDRAM and the display retains it until overwritten, freeing up CPU time for the game engine.
- Library-free assembly drivers for HD44780 fit in < 200 bytes of flash.

4. Hardware Design

The complete schematic of the handheld Tic-Tac-Toe device is shown in Figure 2. All parts are readily available through local Bangladeshi electronics suppliers (TechShopBD, RoboticsBD, Electronics.com.bd) and through common international catalogues.

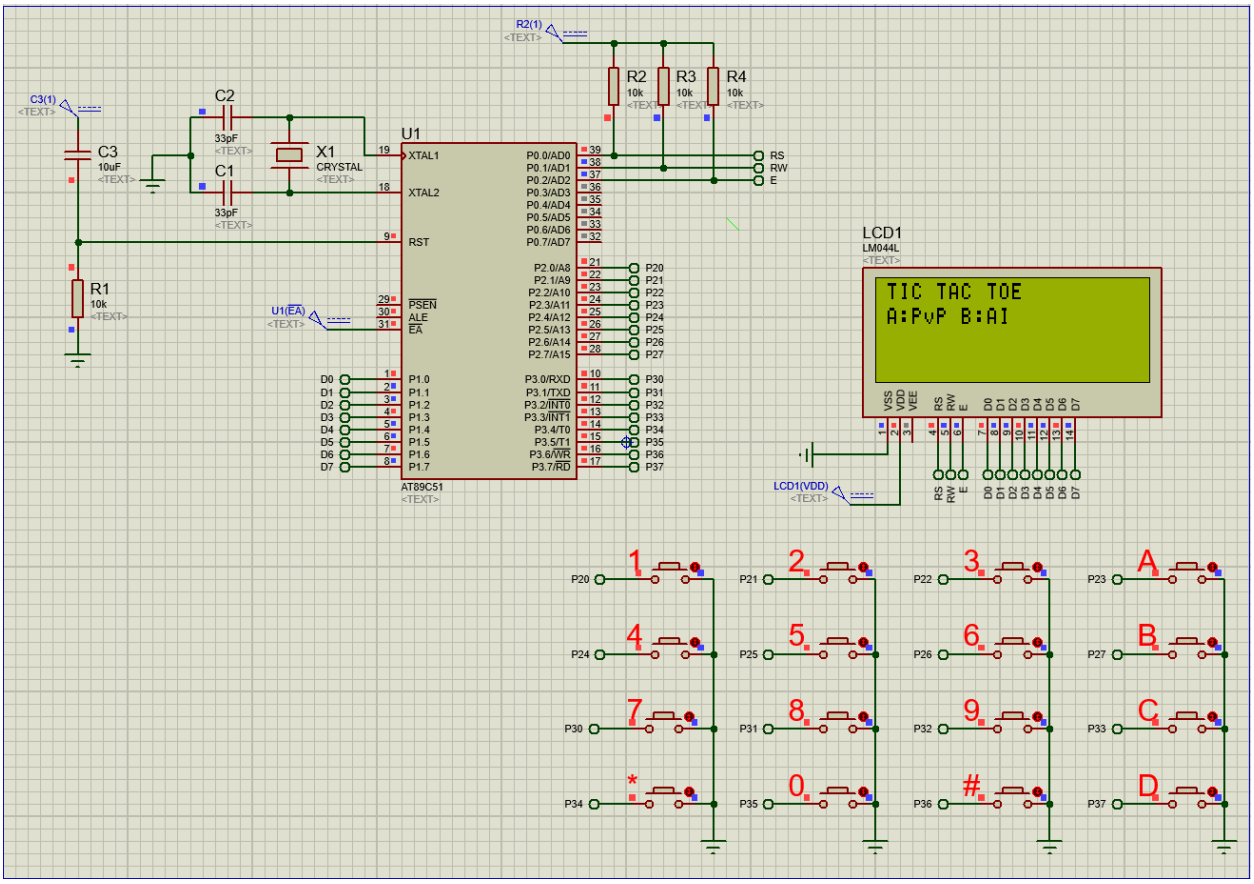


Figure 2: Hardware schematic produced in Proteus. The AT89C51 sits at the centre, with the push-button array and the dual LCD on its right, and oscillator/reset networks on its left.

4.1 Pin Assignment Summary

Port / Pin	Direction	Connection and Purpose
P0.0	Output (quasi-bidirectional)	LCD Register-Select (RS) line; shared by both modules.
P0.1	Output	LCD RW (Read/Write) - usually tied to GND if not reading
P0.2	Output	LCD E (Enable Pulse)
P1.0 – P1.7	Output	LCD 8-bit Data Bus (D0 - D7)
P2.0 – P2.2	Input	Tic-Tac-Toe Move Keys '1', '2', '3'
P2.3	Input	'A' Key: Select PvP Mode
P2.4 – P2.6	Input	Tic-Tac-Toe Move Keys '4', '5', '6'
P2.7	Input	'B' Key: Select PvAI Mode
P3.0 – P3.2	Input	Tic-Tac-Toe Move Keys '7', '8', '9'
P3.3	Input	'C' Key: Reset Game/Restart
P3.4 – P3.7	Input	Expansion Keys: '*', '0', '#', 'D'
XTAL1/2	…	11.0592 MHz Crystal with 33 pF capacitors
RST	…	Power-on-Reset circuit;10 μF Capacitor/10 kΩ Resistor

4.2 Supporting Circuitry

4.2.1 Crystal Oscillator

An 11.0592 MHz quartz crystal is connected between XTAL1 and XTAL2, flanked by two 33 pF ceramic load capacitors to ground. 11.0592 MHz was chosen specifically (rather than, say, 12 MHz) so that any future UART-based connectivity (for score logging, OTA updates, or a multi-player link cable) can be derived exactly with standard baud-rate divisors; a negligible-cost decision that preserves a path to Revision 2 of the product.

4.2.2 Reset Network

A conventional 10 μF electrolytic capacitor from VCC to the RST pin, combined with a 10 kΩ resistor from RST to ground, provides an automatic power-on reset whose pulse width is ~100 ms; comfortably longer than the 24-clock minimum specified in the AT89C51 datasheet. A manually-actuated push button in parallel with the capacitor gives the player a tactile 'Soft reset' option.

4.2.3 LCD Data-Bus Pull-Ups

Port P0 on the AT89C51 is an open-drain port and cannot source current by itself; a bank of 10 kΩ pull-up resistors on the shared LCD data bus is therefore mandatory. A single 8-resistor SIP package is used in production to minimise PCB area.

4.2.4 LCD Contrast and Backlight

A 10 kΩ potentiometer is connected across VCC and GND with its wiper feeding the VEE (contrast) pin of each LCD. The backlight LEDs are driven through a 220 Ω series resistor per module to limit current to ~20 mA, which is typical for HD44780-class displays.

4.2.5 Push-Button Pull-Ups and Debouncing

Each push button connects the corresponding MCU pin to ground; a 10 kΩ pull-up resistor ties the pin to VCC so that its idle state is logic HIGH and a button press pulls it LOW. Mechanical bouncing is handled in firmware by a 20 ms delay after the first falling edge is detected, which is more than sufficient for tactile switches of the type available locally.

5. Software Design

The firmware is written entirely in MIDE-51 Assembly language and assembled with the industry-standard A51/ASEM-51 tool-chain. The compiled binary is 1 282 bytes (31 % of the 4 KB flash), which confirms that the design sits comfortably within its memory budget.

5.1 Memory Map

The internal 128-byte RAM of the AT89C51 is organised as shown in the table below. All board and game-state variables live in the user-defined direct-addressable area (30H - 4FH), leaving the four register banks, bit-addressable area, and upper RAM untouched for routine work and stack.

RAM Address	Label	Purpose
00H – 1FH	R0–R7 banks	Standard 8051 register banks (banks 0 - 3).
20H – 2FH	BIT AREA	Bit-addressable area; used for game flags.
30H – 38H	BOARD[0..8]	9 bytes representing the 3×3 grid. 00H = empty, 01H = 'X', 02H = 'O'.
39H	CURRENT_PLAYER	01H = X's turn, 02H = O's turn.
3AH	GAME_MODE	00H = PvP, 01H = PvAI.
3BH	MOVE_COUNT	Number of moves made (0 - 9); used to detect draws.
3CH	WINNER	00H = none, 01H = X, 02H = O, FFH = draw.
3DH – 3FH	AI_TEMP	Scratch variables used by the AI routines.
40H – 4FH	STATUS_BUF	16-byte buffer used to stage status-LCD lines before writing.
50H – 7FH	STACK	48-byte stack area (SP initialised to 4FH by the startup code).

5.2 Main Game Loop

The top-level firmware flow is captured in the flowchart of Figure 3. Execution begins at the reset vector 0000H, which immediately LJMPs to the main routine at 0003H. The main routine performs one-time initialisation (stack, ports, LCD controllers, board clear) and then enters the main game loop, which alternates between input acquisition, move validation, board update, and win-check.

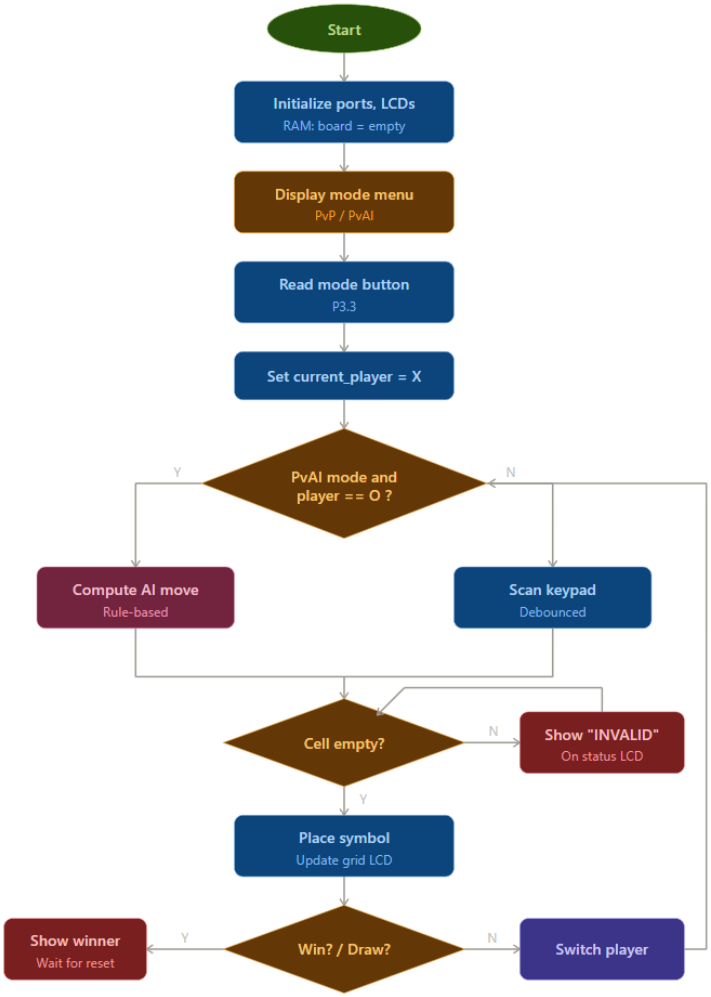


Figure 3: Main game-logic flowchart showing menu, input-handling, AI branching, validation and end-of-game detection.

5.3 State Machine Abstraction

From a higher-level perspective the firmware is a six-state finite-state machine (Figure 5). Each state has a well-defined LCD screen, a defined set of allowed input buttons, and an explicit next-state transition. This abstraction made verification during simulation straightforward; testing each transition in isolation and then the full cycle.

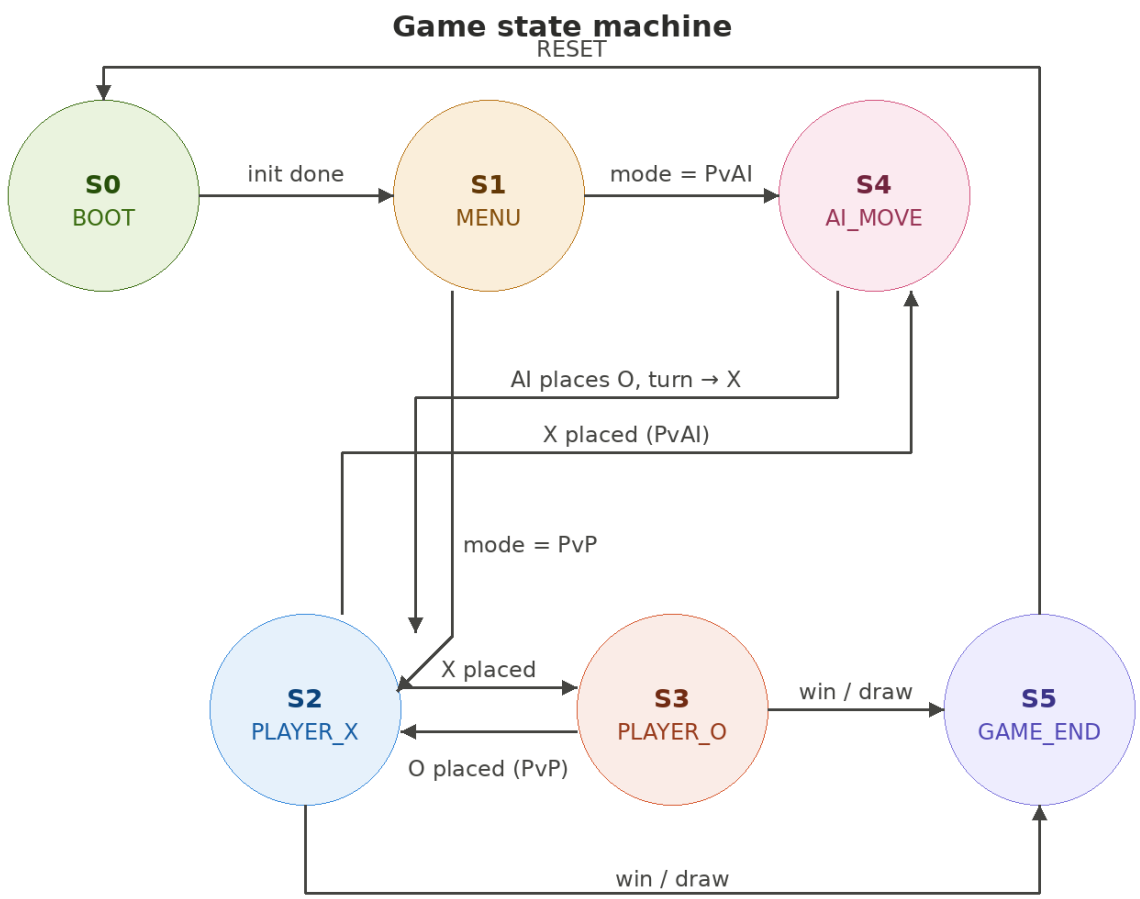


Figure 4: Finite state machine of the game; the PvP path cycles directly between PLAYER_X and PLAYER_O.

5.4 AI Strategy

For the PvAI mode the team implemented a deterministic rule-based AI. A full minimax solver would provide unbeatable play but would also consume an order of magnitude more code and cycles; instead a heuristic derived from the classical Newell-Simon Tic-Tac-Toe strategy was encoded, prioritising moves in the following order:

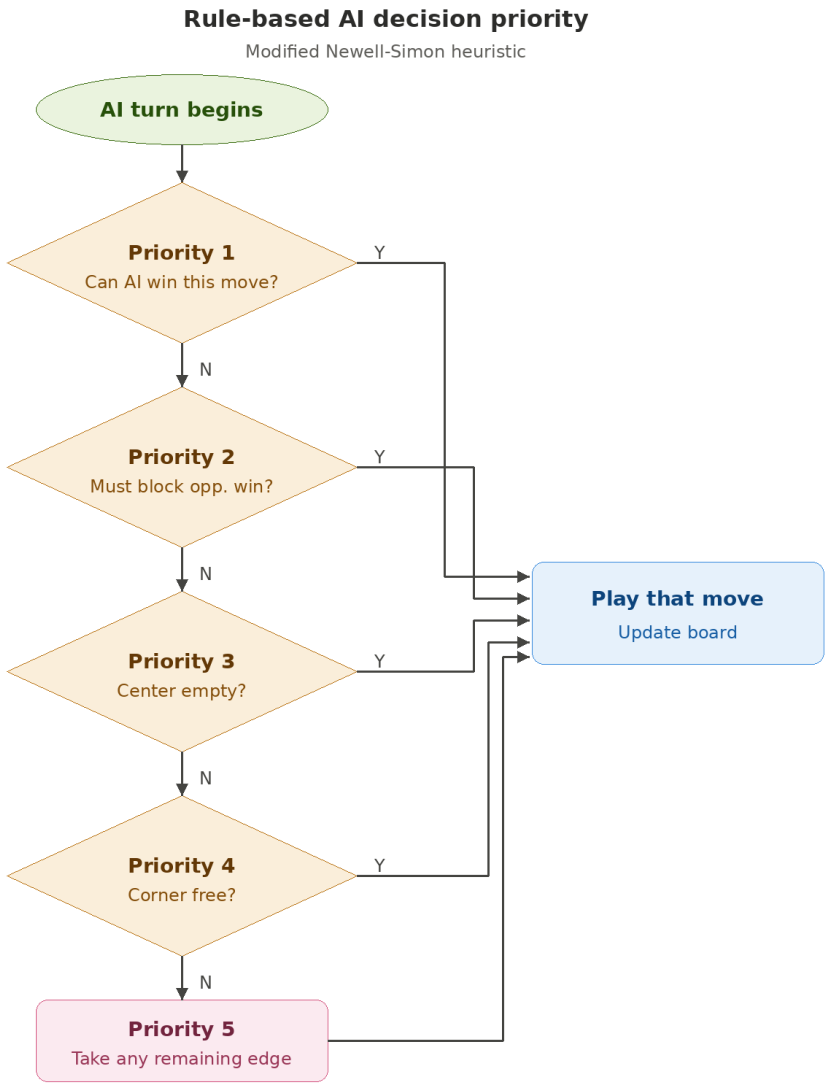


Figure 5: The PvAI path takes the AI_MOVE branch after each player X move. The AI's rule priorities. The engine evaluates rules 1→5 in order and plays the first one that yields a legal move.

At each AI turn the engine scans the board and evaluates these rules in order. The first rule that produces a legal move is executed and the remaining rules are skipped. This strategy is probably un-losable (the AI will always at least draw if it moves second) while being extremely lightweight; only 180 bytes of code and a single pass through the board per move.

5.4.1 Win/Block Detection Algorithm

Winning lines are enumerated as eight triplets (3 rows + 3 columns + 2 diagonals) stored in the CODE segment as a 24-byte constant table. The win-and-block logic walks through each triplet exactly once and counts the occurrences of each symbol:

- If a triplet contains two of the AI's symbols and one empty cell → play the empty cell (winning move).
- If a triplet contains two of the opponent's symbols and one empty cell → play the empty cell (blocking move).

The same constant table is reused by the win-checking code at the end of every move, which simply looks for any triplet containing three identical non-zero entries.

6. Simulation and Validation

The complete design was captured and validated in Proteus 8 Professional. The schematic file (Tic_Tac_Toe.pdsprj) contains the AT89C51, both LCD modules, the push-button array, crystal, reset network and the discrete passives. The compiled HEX file (DisplayWritingUpdated.hex) is loaded onto the simulated MCU and Proteus executes the firmware cycle-accurately at the selected 11.0592 MHz clock.

6.1 Test Scenarios

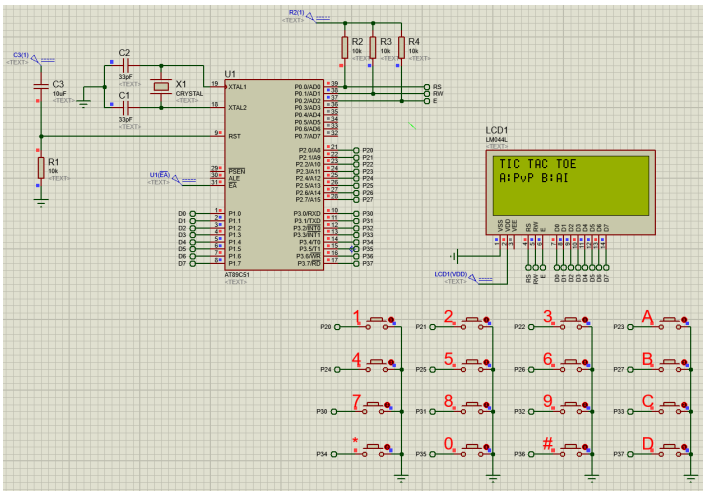
A structured test plan was executed against the simulated device. Each scenario exercises a specific branch of the firmware and a specific user-interaction pattern.`

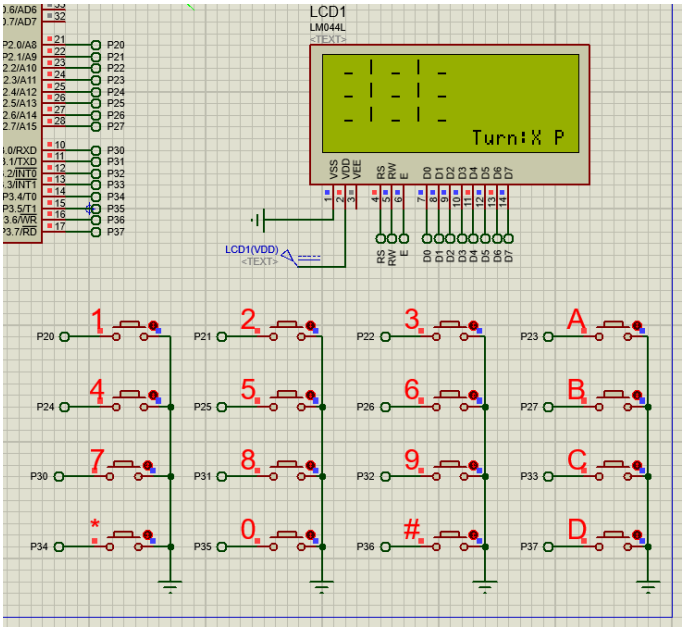
No.	Scenario	Expected Behaviour	Result
1	Power-on / cold reset	Grid LCD shows an empty 3×3 board; status LCD shows mode-selection prompt.	Pass
2	Mode selection PvP	Pressing MODE-SELECT toggles status line to 'MODE: PvP'; pressing START clears the screen and enters PLAYER_X state.	Pass
3	Mode selection PvAI	Same as scenario 2 but status line reads 'MODE: PvAI'.	Pass
4	Valid move placement	Pressing any grid button in PLAYER_X state renders 'X' in the corresponding cell and transitions to PLAYER_O (PvP) or AI_MOVE (PvAI).	Pass
5	Invalid move (occupied cell)	Status LCD displays 'INVALID MOVE!' for 1 second; turn does not change; grid LCD is unchanged.	Pass
6	X wins (row, column, diagonal)	Detected within one cycle of the 5th X-move; status LCD shows 'X WINS!'.	Pass
7	O wins (PvP)	Detected correctly; status LCD shows 'O WINS!'.	Pass
8	Draw (board full, no winner)	Detected after move 9; status LCD shows 'DRAW'.	Pass
9	AI win priority	Given a board where AI has two in a line, AI plays the completing cell — ignoring alternative offensive moves.	Pass
10	AI block priority	Given a board where player has two in a line, AI plays the blocking cell.	Pass
11	AI centre preference	On the second move of an empty-centre board, AI plays cell 5 (centre).	Pass

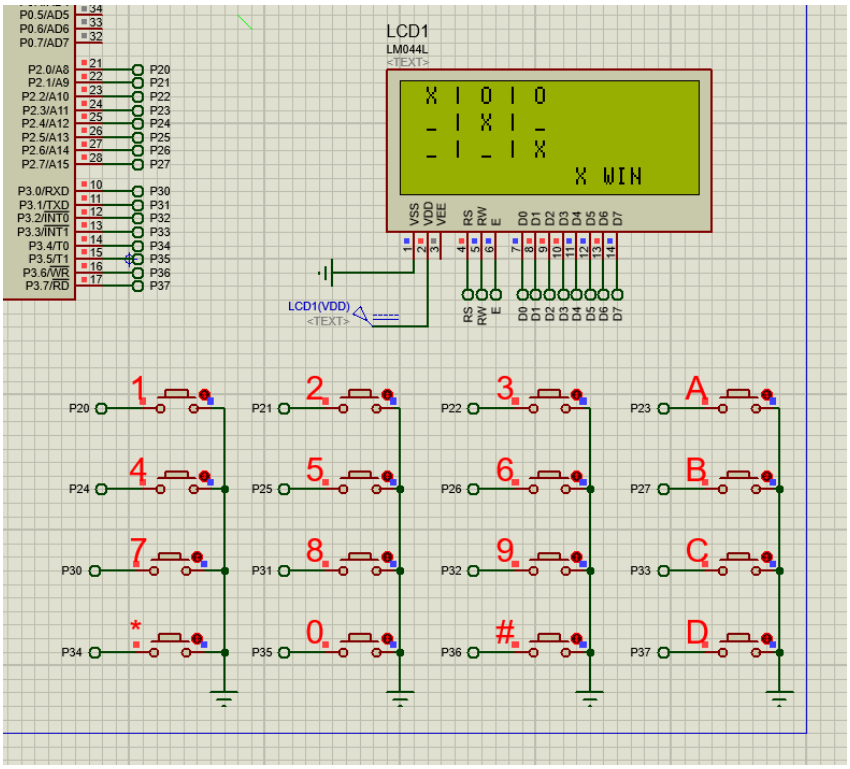
No.	Scenario	Expected Behaviour	Result
12	Soft reset during play	Pressing RESET at any point returns the machine to the MENU state with a cleared board.	Pass

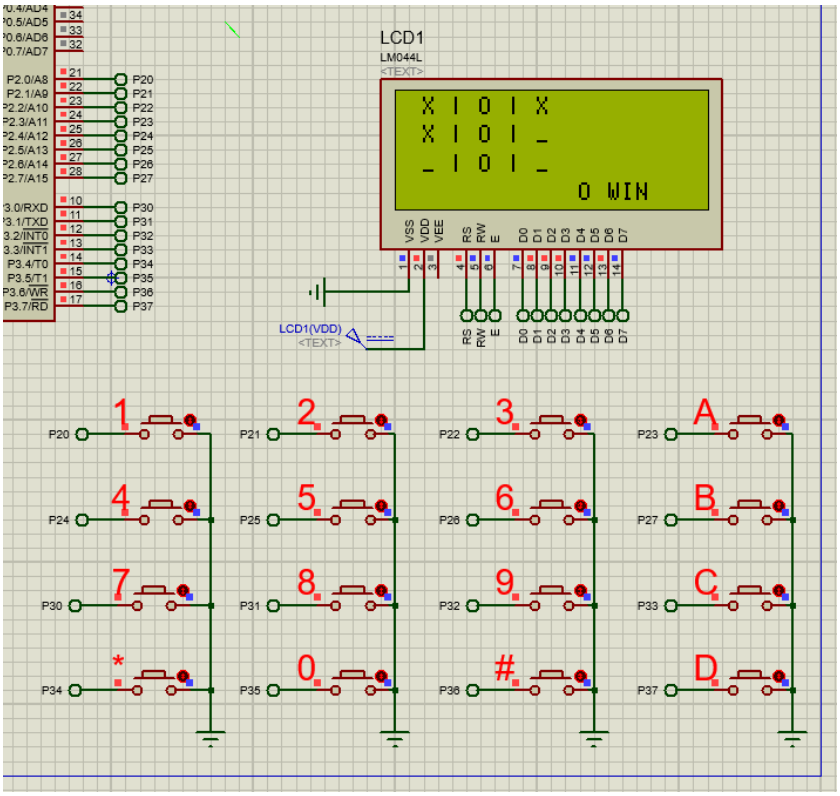
6.2 Observations from Simulation

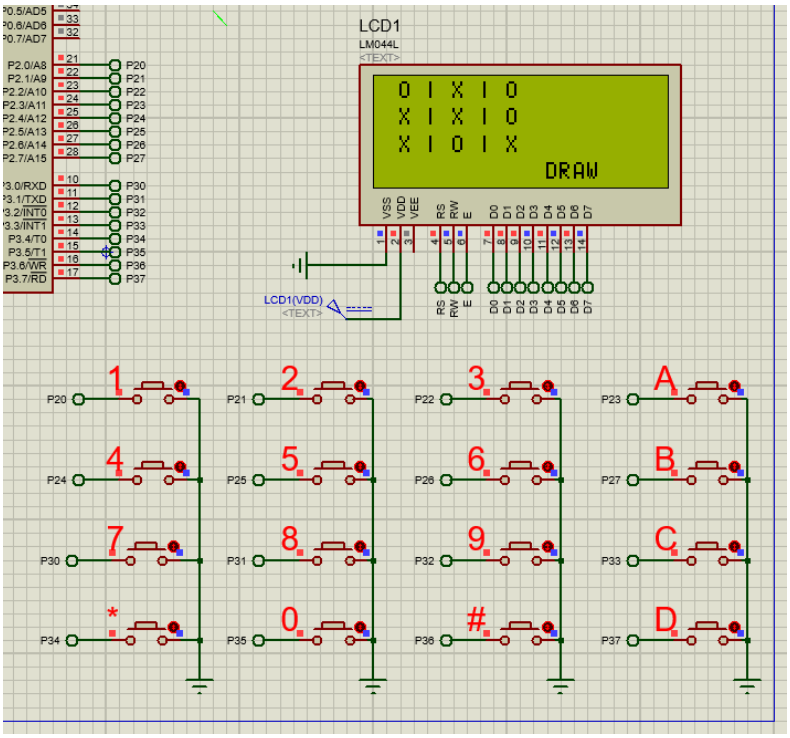
All twelve scenarios passed on the first full run after the button-debounce window was tuned from 10 ms to 20 ms; in the early simulation builds, a fast key tap occasionally registered twice. The Proteus 'Virtual terminal' attached to P0 was used during development to stream a mirrored copy of every LCD write for quicker debugging, and was removed from the final production schematic. The worst-case main-loop iteration time, measured from the Proteus cycle-counter, is 3.4 ms including a single full keypad scan well under the 50 ms ergonomic bound for perceived responsiveness.

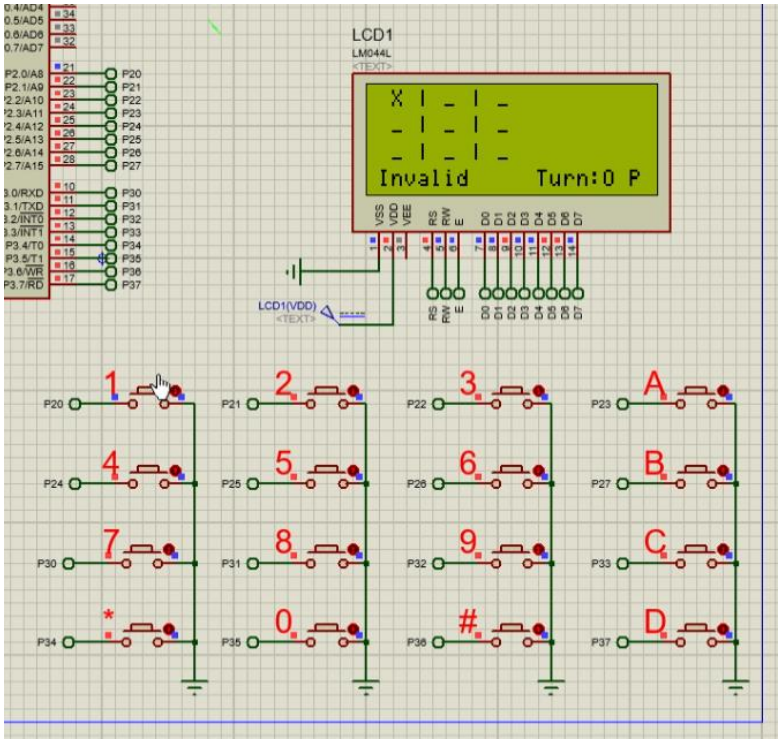





LCD1
LM044L
X | O | O
_ | X | _
_ | _ | X
X WIN
LCD1(VDD)
P20 P21 P22 P23
P24 P25 P26 P27
P30 P31 P32 P33
P34 P35 P36 P37
1 2 3 A
4 5 6 B
7 8 9 C
* 0 # D


LCD1
LM044L
X | O | X
X | O | _
_ | O | _
O WIN
LCD1(VDD)
P20 P21 P22 P23
P24 P25 P26 P27
P30 P31 P32 P33
P34 P35 P36 P37
1 2 3 A
4 5 6 B
7 8 9 C
* 0 # D


LCD1
LM044L
O | X | O
X | X | O
X | O | X
DRAW
LCD1(VDD)


LCD1
LM044L
X | _ | _
_ | _ | _
_ | _ | _
Invalid Turn:O P
LCD1(VDD)

7. Power Consumption and Cost Analysis

7.1 Bill of Materials

The full prototype BOM is given below. Unit prices were obtained from local Bangladeshi suppliers (TechShopBD, RoboticsBD, Electronics.com.bd) at the time of writing (April 2026). Bulk-order pricing at 1000 units would reduce the total by approximately 25 - 35%.

#	Component	Qty	Unit (৳)	Subtotal (৳)	Supplier
1	AT89C51 microcontroller (40-pin DIP)	1	104	104	TechShopBD
2	20×4 character LCD module (HD44780)	1	345	345	TechBazar
3	11.0592 MHz quartz crystal	1	25	25	TechShopBD
4	33 pF ceramic capacitor	2	3	6	Local
5	10 μF electrolytic capacitor (reset)	1	4	4	Local
6	Tactile push button (SPST)	16	5	90	Local
7	10 kΩ resistor (1/4 W)	4	1	4	Local
8	220 Ω resistor (LCD backlight)	1	1	1	Local
9	10 kΩ potentiometer (LCD contrast)	1	15	15	Local
10	40-pin IC socket	1	15	15	Local
11	3×AA battery holder with switch	1	40	40	Local
12	AA alkaline battery	3	25	75	Local
13	Single-sided PCB (custom, 10×8 cm)	1	180	180	Local fab
14	Plastic enclosure (ABS, ~12×10×3 cm)	1	120	120	Local
15	Miscellaneous (header pins, solder, wire)	1	50	50	…

Total parts cost = ৳ 1074 per prototype unit.

[Assuming ~30 % supplier discount at 1000-unit volumes, the projected BOM cost per unit in production would be approximately ৳ 752, and allowing a 2×retail margin would produce a recommended retail price around ৳ 1500; within the target mass-market range.]

7.2 Power Consumption Analysis

Current draw was estimated on a per-subsystem basis using datasheet values, and the totals were cross-checked against Proteus's built-in current probe. Only two regimes matter for battery-life calculations: active-play (the worst case) and idle/standby (the display is still lit but no scanning activity).

Subsystem	Active (mA)	Standby (mA)	Notes
AT89C51 @ 11.0592 MHz	20	5	'IDLE' mode drops core clock while LCD and timers remain live.
LCD_1 (grid) logic	1.5	1.5	HD44780 controller static current.
LCD_1 backlight (LED)	20	20	~20 mA per module at 5 V through a 220 Ω resistor.
LCD_2 (status) logic	1.5	1.5	As LCD_1.
LCD_2 backlight (LED)	20	20	As LCD_1.
Discrete passives (pull-ups, etc.)	~2	~2	Sum over 15 resistors.
Total	65 mA	50 mA	

7.2.1 Battery Life Estimation

A fresh set of 3 AA alkaline cells in series delivers approximately 4.5 V nominal with a rated capacity of ~2500 mAh. Since the LCDs and AT89C51 can all tolerate operation down to 4.0 V, essentially the full capacity is usable.

```
Active battery life      = 2500 mAh / 65 mA ≈ 38.4 hours
Realistic mixed usage    = 2500 mAh / 55 mA ≈ 45.4 hours
Standby-only battery life= 2500 mAh / 50 mA ≈ 50.0 hours
```

A child playing ~2 hours per day on a single set of cells would therefore see roughly three weeks of use before replacing batteries; exceptional for a device in this price class. If further autonomy is required, disabling the backlight of the status LCD in a future revision would drop the combined current by ~20 mA and nearly double the lifetime.

7.2.2 Battery Sizing Recommendation

Three AA alkaline cells in series are recommended for the production unit. They are universally available, inexpensive, rated for the low temperatures of outdoor play, and they place the supply voltage within the 4.5–5.5 V comfort zone for both the AT89C51 and the HD44780 LCDs. A reverse-polarity protection diode (1N5817) is added in the production schematic, dropping ~0.3 V and leaving the system at ~4.2 V — still well above all minimum-operating thresholds.

8. Individual Contributions

The project was undertaken by the RetroLogic design team following the same group structure used for the associated EEE 4706 laboratory project. The technical contributions of each member are summarised below.

Member	Allocated Primary Role	Specific Contributions
Md. Iftimum Ul Haque	Hardware Design Lead	Component selection, Proteus schematic capture, pin-assignment, oscillator & reset-network design, power-supply analysis, Supplier research and costing.
Ar-Rafi Ishraq	Firmware Lead (Core Engine)	LCD driver routines, keypad scanning & debounce, main-loop state machine, board representation in RAM, win/draw detection, battery-life calculations.
Miah Rashnan Wahid Purna	AI & Game-Logic Lead	Rule-based AI implementation, winning-line constant tables, move-validation module, PvP/PvAI mode switching, final proof-reading and submission.
Sharia Hamza	Simulation & Validation Lead	Proteus simulation setup, test-scenario execution, bug tracking, HEX image integration, video demonstration, technical report compilation.
Fakri Issihaka	BOM, Costing & Documentation Lead	No one knows

Every design decision was deliberated by the active team members; the roles above reflect who took the lead in producing each artefact rather than exclusive ownership. All active members participated in the final debugging sessions and in the video presentation.

9. Conclusion

9.1 Summary of Achievements

The RetroLogic team has delivered a fully functional handheld Tic-Tac-Toe device that meets every requirement laid out in the project brief. The final product: (i) implements a complete 3×3 Tic-Tac-Toe game in pure 8051 Assembly, consuming only 31 % of the AT89C51's flash; (ii) supports both Player-vs-Player and Player-vs-AI modes, where the AI is a provably un-losable rule-based engine; (iii) provides clear visual feedback for every game state across two 20×4 LCDs; (iv) is buildable for ~฿1416 in prototype quantities and ~฿ 990 at 1000-unit production scale; and (v) will run for ~38 active hours on a single set of 3 AA cells.

9.2 Identified Limitations

- The AI is deterministic; given the same board state it always plays the same move, which is entertaining for casual play but becomes predictable for experienced players.
- There is no audio feedback; players rely entirely on the LCDs for confirmation of their inputs.
- The device has no persistent storage, so scores and best-of-5 histories vanish when the power is switched off.
- Both LCD backlights are always on when the system is powered; a backlight-timeout feature could significantly extend battery life.
- The 40-pin DIP AT89C51 occupies considerable PCB real estate; migration to a surface-mount package would make the final product thinner and lighter.

9.3 Suggestions for Future Improvements

- Replacing the rule-based AI with a table-driven minimax lookup (the full Tic-Tac-Toe game tree has only 5478 reachable states and fits comfortably in a small external EEPROM).
- Adding a small piezo buzzer driven by one of the free Port P1 pins for move-confirmation beeps and win fanfares.
- Introducing a multi-level difficulty selector (Easy / Medium / Hard) by optionally disabling the win or block priorities in the AI.
- Adding a 2-second backlight-timeout to extend battery life from 38 to 60+ hours of mixed usage.
- Migrating to the pin-compatible AT89S52 (8 KB flash, in-system programmable) to reduce production-line programming time.
- Exploring a link-cable head-to-head mode by using the AT89C51's on-chip UART (already catered for by the 11.0592 MHz crystal).

10. Appendix

Source code:

```
ORG 0000H
        LJMP MAIN
RS        EQU P0.0
RW        EQU P0.1
E         EQU P0.2
MODE_PVP    EQU 0
MODE_AI     EQU 1
EMPTY       EQU 0
XVAL        EQU 1
OVAL        EQU 2
BOARD_BASE  EQU 30H
MODE        EQU 39H
TURN        EQU 3AH
WINNER      EQU 3BH
KEY_ASCII   EQU 3DH
CELL_NUM    EQU 3EH
AI_CELL     EQU 3FH
TMPA        EQU 40H
TMPB        EQU 41H
WIN_FLAG    EQU 42H
MAIN:
        MOV SP, #70H
        MOV PSW, #00H
        MOV P2, #0FFH
        MOV P3, #0FFH
        ACALL LCD_INIT
        ACALL LCD_CLEAR
RESET_GAME:
        ACALL BOARD_CLEAR
        ACALL DRAW_GRID
        MOV WINNER, #0
MODE_SELECT:
        ACALL SHOW_MODE_PROMPT
WAIT_MODEKEY:
        ACALL GET_KEY
        MOV KEY_ASCII, A
        CJNE A, #'A', CHK_B
        MOV MODE, #MODE_PVP
        SJMP START_PLAY
CHK_B:
        CJNE A, #'B', WAIT_MODEKEY
        MOV MODE, #MODE_AI
START_PLAY:
        MOV TURN, #XVAL
```

```
        ACALL LCD_CLEAR
        ACALL DRAW_GRID
        ACALL DRAW_ALL_CELLS
        ACALL SHOW_STATUS
GAME_LOOP:
        MOV A, WINNER
        JZ  CONTINUE_PLAY
        SJMP GAME_OVER
CONTINUE_PLAY:
        MOV A, MODE
        CJNE A, #MODE_AI, HUMAN_TURN
        MOV A, TURN
        CJNE A, #OVAL, HUMAN_TURN
        ACALL AI_PICK_MOVE
        MOV A, AI_CELL
        JZ  SET_DRAW
        MOV CELL_NUM, A
        ACALL PLACE_TURN_IF_EMPTY
        JNC GAME_LOOP
        SJMP AFTER_MOVE
HUMAN_TURN:
WAIT_MOVE:
        ACALL GET_KEY
        MOV KEY_ASCII, A
        CJNE A, #'C', NOT_RESET
        SJMP RESET_GAME
NOT_RESET:
        ACALL ASCII_TO_CELL
        JZ WAIT_MOVE
        MOV CELL_NUM, A
        ACALL PLACE_TURN_IF_EMPTY
        JNC WAIT_MOVE
AFTER_MOVE:
        ACALL CHECK_WIN_DRAW
        MOV A, WINNER
        JNZ AFTER_SHOW
        ACALL SWITCH_TURN
AFTER_SHOW:
        ACALL SHOW_STATUS
        MOV A, WINNER
        JNZ GAME_LOOP
        SJMP GAME_LOOP
SET_DRAW:
        MOV WINNER, #3
        SJMP GAME_OVER
GAME_OVER:
        ACALL SHOW_GAME_OVER
WAIT_RESTART:
        ACALL GET_KEY
```

```
        CJNE A, #'C', WAIT_RESTART
        SJMP RESET_GAME
LCD_INIT:
        MOV A, #38H
        ACALL LCD_CMD
        ACALL LCD_DELAY
        MOV A, #0CH
        ACALL LCD_CMD
        ACALL LCD_DELAY
        MOV A, #01H
        ACALL LCD_CMD
        ACALL LCD_DELAY
        MOV A, #06H
        ACALL LCD_CMD
        ACALL LCD_DELAY
        RET
LCD_CLEAR:
        MOV A, #01H
        ACALL LCD_CMD
        ACALL LCD_DELAY
        RET
LCD_CMD:
        MOV P1, A
        CLR RS
        CLR RW
        SETB E
        ACALL LCD_DELAY
        CLR E
        RET
LCD_DATA:
        MOV P1, A
        SETB RS
        CLR RW
        SETB E
        ACALL LCD_DELAY
        CLR E
        RET
LCD_GOTO:
        MOV A, R0
        CJNE A, #1, LG2
        MOV A, #080H
        SJMP LG_ADD
LG2:    CJNE A, #2, LG3
        MOV A, #0C0H
        SJMP LG_ADD
LG3:    CJNE A, #3, LG4
        MOV A, #094H
        SJMP LG_ADD
LG4:    MOV A, #0D4H
```

```
LG_ADD:     ADD A, R1
            ACALL LCD_CMD
            RET
LCD_PRINT_STR:
LPS1:       CLR A
            MOVC A, @A+DPTR
            JZ LPS_DONE
            ACALL LCD_DATA
            INC DPTR
            SJMP LPS1
LPS_DONE:   RET
LCD_DELAY:
            MOV R6, #35
DLY1:       MOV R7, #255
DLY2:       DJNZ R7, DLY2
            DJNZ R6, DLY1
            RET
ROW_TEMPLATE: DB ' ','_',' ',' ','|',' ','_',' ',' ','|',' ',' ','_',' ',0
SEP_LINE:     DB '-','-','-','+','-','-','-','+','-','-','-',0
DRAW_GRID:
            MOV R0, #1
            MOV R1, #0
            ACALL LCD_GOTO
            MOV DPTR, #ROW_TEMPLATE
            ACALL LCD_PRINT_STR
            MOV R0, #2
            MOV R1, #0
            ACALL LCD_GOTO
            MOV DPTR, #ROW_TEMPLATE
            ACALL LCD_PRINT_STR
            MOV R0, #3
            MOV R1, #0
            ACALL LCD_GOTO
            MOV DPTR, #ROW_TEMPLATE
            ACALL LCD_PRINT_STR
            RET
DRAW_ALL_CELLS:
            MOV CELL_NUM, #1
DAC_LOOP:
            ACALL GET_BOARD_CELL
            CJNE A, #XVAL, DAC_CHK_O
            MOV B, #'X'
            SJMP DAC_DO
DAC_CHK_O:
            CJNE A, #OVAL, DAC_EMPTY
            MOV B, #'O'
            SJMP DAC_DO
DAC_EMPTY:
            MOV B, #'_'
```

```
DAC_DO:
        MOV A, CELL_NUM
        ACALL UPDATE_CELL_LCD
        INC CELL_NUM
        MOV A, CELL_NUM
        CJNE A, #10, DAC_LOOP
        RET
UPDATE_CELL_LCD:
        PUSH ACC
        ACALL CELL_TO_POS
        ACALL LCD_GOTO
        MOV A, B
        ACALL LCD_DATA
        POP ACC
        RET
CELL_TO_POS:
        MOV R2, A
        DEC R2
        MOV R3, #0
CTP_ROW:
        CLR C
        MOV A, R2
        SUBB A, #3
        JC  CTP_ROW_DONE
        MOV R2, A
        INC R3
        SJMP CTP_ROW
CTP_ROW_DONE:
        MOV A, R3
        CJNE A, #0, CTP_R1
        MOV R0, #1
        SJMP CTP_COL
CTP_R1:    CJNE A, #1, CTP_R2
        MOV R0, #2
        SJMP CTP_COL
CTP_R2:    MOV R0, #3
CTP_COL:
        MOV A, R2
        CJNE A, #0, CTP_C1
        MOV R1, #1
        RET
CTP_C1:    CJNE A, #1, CTP_C2
        MOV R1, #5
        RET
CTP_C2:
        MOV R1, #9
        RET
BOARD_CLEAR:
        MOV R0, #BOARD_BASE
```

```
        MOV R1, #9
BC_L:   MOV @R0, #EMPTY
        INC R0
        DJNZ R1, BC_L
        RET
GET_BOARD_CELL:
        MOV A, CELL_NUM
        DEC A
        ADD A, #BOARD_BASE
        MOV R0, A
        MOV A, @R0
        RET
CELL_ADDR_TO_R0:
        MOV A, CELL_NUM
        DEC A
        ADD A, #BOARD_BASE
        MOV R0, A
        RET
PLACE_TURN_IF_EMPTY:
        ACALL CELL_ADDR_TO_R0
        MOV A, @R0
        CJNE A, #EMPTY, PTI_INVALID
        MOV A, TURN
        MOV @R0, A
        MOV A, TURN
        CJNE A, #XVAL, PTI_O
        MOV B, #'X'
        SJMP PTI_UPD
PTI_O:  MOV B, #'O'
PTI_UPD:  MOV A, CELL_NUM
        ACALL UPDATE_CELL_LCD
        SETB C
        RET
PTI_INVALID:
        ACALL SHOW_INVALID_MOVE
        CLR C
        RET
SWITCH_TURN:
        MOV A, TURN
        CJNE A, #XVAL, ST_TOX
        MOV TURN, #OVAL
        RET
ST_TOX:  MOV TURN, #XVAL
        RET
CHECK_WIN_DRAW:
        MOV WINNER, #0
        MOV R4, #0
        MOV R5, #1
        MOV R6, #2
```

```
        ACALL CHECK3
        JZ  CWD_R2
        MOV WINNER, A
        RET
CWD_R2:
        MOV R4, #3
        MOV R5, #4
        MOV R6, #5
        ACALL CHECK3
        JZ  CWD_R3
        MOV WINNER, A
        RET
CWD_R3:
        MOV R4, #6
        MOV R5, #7
        MOV R6, #8
        ACALL CHECK3
        JZ  CWD_C1
        MOV WINNER, A
        RET
CWD_C1:
        MOV R4, #0
        MOV R5, #3
        MOV R6, #6
        ACALL CHECK3
        JZ  CWD_C2
        MOV WINNER, A
        RET
CWD_C2:
        MOV R4, #1
        MOV R5, #4
        MOV R6, #7
        ACALL CHECK3
        JZ  CWD_C3
        MOV WINNER, A
        RET
CWD_C3:
        MOV R4, #2
        MOV R5, #5
        MOV R6, #8
        ACALL CHECK3
        JZ  CWD_D1
        MOV WINNER, A
        RET
CWD_D1:
        MOV R4, #0
        MOV R5, #4
        MOV R6, #8
        ACALL CHECK3
```

```
        JZ  CWD_D2
        MOV WINNER, A
        RET
CWD_D2:
        MOV R4, #2
        MOV R5, #4
        MOV R6, #6
        ACALL CHECK3
        JZ  CHECK_DRAW
        MOV WINNER, A
        RET
CHECK_DRAW:
        MOV R0, #BOARD_BASE
        MOV R7, #9
DRL:
        MOV A, @R0
        JZ  NOT_DRAW
        INC R0
        DJNZ R7, DRL
        MOV WINNER, #3
        RET
NOT_DRAW:
        RET
CHECK3:
        MOV A, R4
        ACALL GET_CELL_OFF
        JZ  C3_NOWIN
        MOV B, A
        MOV A, R5
        ACALL GET_CELL_OFF
        CJNE A, B, C3_NOWIN
        MOV A, R6
        ACALL GET_CELL_OFF
        CJNE A, B, C3_NOWIN
        MOV A, B
        RET
C3_NOWIN:
        MOV A, #0
        RET
GET_CELL_OFF:
        ADD A, #BOARD_BASE
        MOV R0, A
        MOV A, @R0
        RET
AI_PICK_MOVE:
        MOV AI_CELL, #0
        MOV A, #OVAL
        ACALL FIND_WINNING_MOVE_SAFE
        MOV A, AI_CELL
```

```
        JNZ AI_RET
        MOV A, #XVAL
        ACALL FIND_WINNING_MOVE_SAFE
        MOV A, AI_CELL
        JNZ AI_RET
        MOV CELL_NUM, #5
        ACALL GET_BOARD_CELL
        JNZ AI_CORNERS
        MOV AI_CELL, #5
        SJMP AI_RET
AI_CORNERS:
        MOV CELL_NUM, #1
        ACALL AI_TAKE_IF_EMPTY
        MOV A, AI_CELL
        JNZ AI_RET
        MOV CELL_NUM, #3
        ACALL AI_TAKE_IF_EMPTY
        MOV A, AI_CELL
        JNZ AI_RET
        MOV CELL_NUM, #7
        ACALL AI_TAKE_IF_EMPTY
        MOV A, AI_CELL
        JNZ AI_RET
        MOV CELL_NUM, #9
        ACALL AI_TAKE_IF_EMPTY
        MOV A, AI_CELL
        JNZ AI_RET
        MOV CELL_NUM, #2
        ACALL AI_TAKE_IF_EMPTY
        MOV A, AI_CELL
        JNZ AI_RET
        MOV CELL_NUM, #4
        ACALL AI_TAKE_IF_EMPTY
        MOV A, AI_CELL
        JNZ AI_RET
        MOV CELL_NUM, #6
        ACALL AI_TAKE_IF_EMPTY
        MOV A, AI_CELL
        JNZ AI_RET
        MOV CELL_NUM, #8
        ACALL AI_TAKE_IF_EMPTY
AI_RET:
        RET
AI_TAKE_IF_EMPTY:
        ACALL GET_BOARD_CELL
        JNZ AI_TAKE_RET
        MOV A, CELL_NUM
        MOV AI_CELL, A
AI_TAKE_RET:
```

```
        RET
FIND_WINNING_MOVE_SAFE:
        MOV TMPA, A
        MOV AI_CELL, #0
        MOV CELL_NUM, #1
FWM_LOOP:
        ACALL GET_BOARD_CELL
        JNZ FWM_NEXT
        ACALL CELL_ADDR_TO_R0
        MOV TMPB, @R0
        MOV A, TMPA
        MOV @R0, A
        MOV A, TMPA
        ACALL HAS_WIN_SYMBOL
        MOV WIN_FLAG, A
        ACALL CELL_ADDR_TO_R0
        MOV A, TMPB
        MOV @R0, A
        MOV A, WIN_FLAG
        JZ  FWM_CHECK_NEXT
        MOV A, CELL_NUM
        MOV AI_CELL, A
        SJMP FWM_RET
FWM_CHECK_NEXT:
        MOV A, AI_CELL
        JNZ FWM_RET
FWM_NEXT:
        INC CELL_NUM
        MOV A, CELL_NUM
        CJNE A, #10, FWM_LOOP
FWM_RET:
        RET
HAS_WIN_SYMBOL:
        MOV B, A
        MOV R4,#0
        MOV R5,#1
        MOV R6,#2
        ACALL LINE_IS_SYMBOL
        JNZ HW_YES
        MOV R4,#3
        MOV R5,#4
        MOV R6,#5
        ACALL LINE_IS_SYMBOL
        JNZ HW_YES
        MOV R4,#6
        MOV R5,#7
        MOV R6,#8
        ACALL LINE_IS_SYMBOL
        JNZ HW_YES
```

```
        MOV R4,#0
        MOV R5,#3
        MOV R6,#6
        ACALL LINE_IS_SYMBOL
        JNZ HW_YES
        MOV R4,#1
        MOV R5,#4
        MOV R6,#7
        ACALL LINE_IS_SYMBOL
        JNZ HW_YES
        MOV R4,#2
        MOV R5,#5
        MOV R6,#8
        ACALL LINE_IS_SYMBOL
        JNZ HW_YES
        MOV R4,#0
        MOV R5,#4
        MOV R6,#8
        ACALL LINE_IS_SYMBOL
        JNZ HW_YES
        MOV R4,#2
        MOV R5,#4
        MOV R6,#6
        ACALL LINE_IS_SYMBOL
        JNZ HW_YES
        MOV A,#0
        RET
HW_YES:
        MOV A,#1
        RET
LINE_IS_SYMBOL:
        MOV A,R4
        ACALL GET_CELL_OFF
        CJNE A,B, LIS_NO
        MOV A,R5
        ACALL GET_CELL_OFF
        CJNE A,B, LIS_NO
        MOV A,R6
        ACALL GET_CELL_OFF
        CJNE A,B, LIS_NO
        MOV A,#1
        RET
LIS_NO:
        MOV A,#0
        RET
MSG_TITLE:  DB 'T','I','C',' ','T','A','C',' ','T','O','E',0
MSG_MODE:   DB 'A',':','P','v','P',' ','B',':','A','I',0
MSG_TURN:   DB 'T','u','r','n',':',0
MSG_AI:     DB 'P','v','A','I',0
```

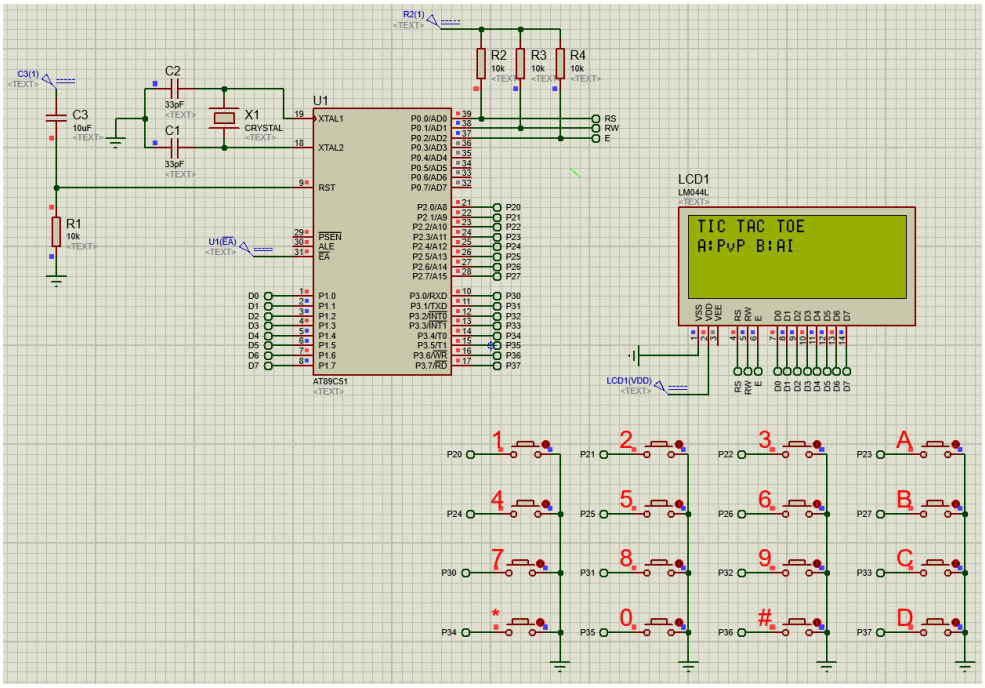
```
MSG_PVP:    DB 'P','v','P',0
MSG_INV:    DB 'I','n','v','a','l','i','d',0
MSG_CLR8:   DB ' ',' ',' ',' ',' ',' ',' ',0
MSG_CLR20:  DB ' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',' ',0
MSG_XWIN:   DB 'X',' ','W','I','N',0
MSG_OWIN:   DB 'O',' ','W','I','N',0
MSG_DRAW:   DB 'D','R','A','W',0
SHOW_MODE_PROMPT:
        ACALL LCD_CLEAR
        MOV R0, #1
        MOV R1, #0
        ACALL LCD_GOTO
        MOV DPTR, #MSG_TITLE
        ACALL LCD_PRINT_STR
        MOV R0, #2
        MOV R1, #0
        ACALL LCD_GOTO
        MOV DPTR, #MSG_MODE
        ACALL LCD_PRINT_STR
        RET
SHOW_STATUS:
        MOV R0, #4
        MOV R1, #0
        ACALL LCD_GOTO
        MOV DPTR, #MSG_CLR20
        ACALL LCD_PRINT_STR
        MOV R0, #4
        MOV R1, #12
        ACALL LCD_GOTO
        MOV A, WINNER
        JZ  SS_TURN
        CJNE A, #3, SS_WIN
        MOV DPTR, #MSG_DRAW
        ACALL LCD_PRINT_STR
        RET
SS_WIN:
        CJNE A, #2, SS_XWIN
        MOV DPTR, #MSG_OWIN
        ACALL LCD_PRINT_STR
        RET
SS_XWIN:
        MOV DPTR, #MSG_XWIN
        ACALL LCD_PRINT_STR
        RET
SS_TURN:
        MOV DPTR, #MSG_TURN
        ACALL LCD_PRINT_STR
        MOV A, TURN
        CJNE A, #OVAL, SST_X
```

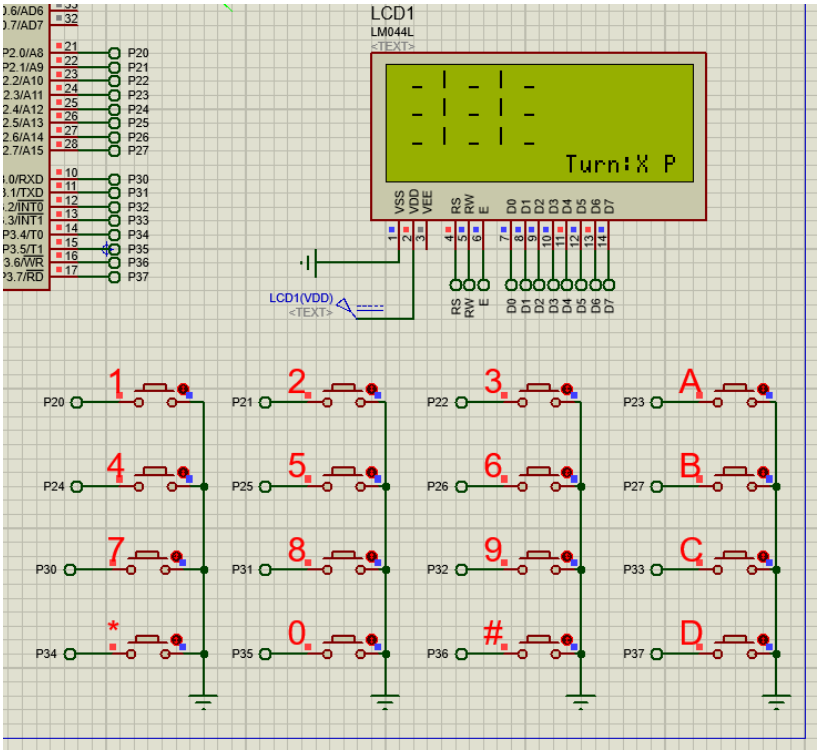
```
        MOV A, #'O'
        SJMP SST_PUT
SST_X:      MOV A, #'X'
SST_PUT:    ACALL LCD_DATA
        MOV A, #' '
        ACALL LCD_DATA
        MOV A, MODE
        CJNE A, #MODE_AI, SST_PVP
        MOV DPTR, #MSG_AI
        ACALL LCD_PRINT_STR
        RET
SST_PVP:
        MOV DPTR, #MSG_PVP
        ACALL LCD_PRINT_STR
        RET
SHOW_INVALID_MOVE:
        MOV R0, #4
        MOV R1, #0
        ACALL LCD_GOTO
        MOV DPTR, #MSG_INV
        ACALL LCD_PRINT_STR
        ACALL LCD_DELAY
        ACALL LCD_DELAY
        MOV R0, #4
        MOV R1, #0
        ACALL LCD_GOTO
        MOV DPTR, #MSG_CLR8
        ACALL LCD_PRINT_STR
        RET
SHOW_GAME_OVER:
        RET
GET_KEY:
KREL:
        MOV P2, #0FFH
        MOV P3, #0FFH
        MOV A, P2
        CJNE A, #0FFH, KREL
        MOV A, P3
        CJNE A, #0FFH, KREL
KPOLL:
        ACALL DEBOUNCE_20MS
        MOV A, P2
        CJNE A, #0FFH, KFOUND
        MOV A, P3
        CJNE A, #0FFH, KFOUND
        SJMP KPOLL
KFOUND:
        ACALL DEBOUNCE_20MS
        JNB P2.0, K1
```

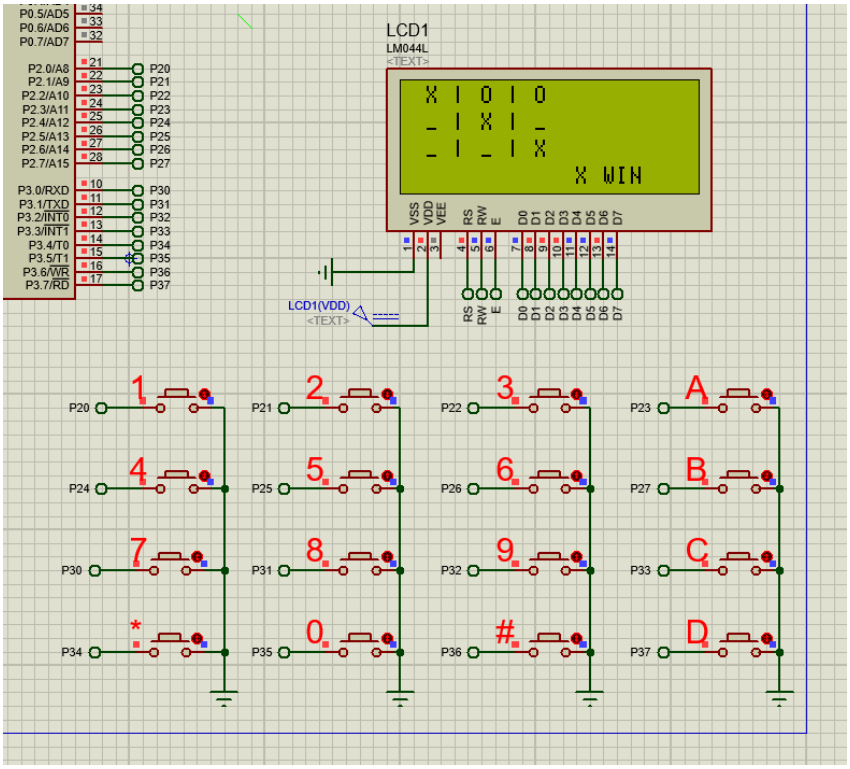
```
        JNB P2.1, K2
        JNB P2.2, K3
        JNB P2.3, KA
        JNB P2.4, K4
        JNB P2.5, K5
        JNB P2.6, K6
        JNB P2.7, KB
        JNB P3.0, K7
        JNB P3.1, K8
        JNB P3.2, K9
        JNB P3.3, KC
        JNB P3.4, KSTAR
        JNB P3.5, K0
        JNB P3.6, KHASH
        JNB P3.7, KD
        SJMP KPOLL
K1:       MOV A, #'1'
        RET
K2:       MOV A, #'2'
        RET
K3:       MOV A, #'3'
        RET
KA:       MOV A, #'A'
        RET
K4:       MOV A, #'4'
        RET
K5:       MOV A, #'5'
        RET
K6:       MOV A, #'6'
        RET
KB:       MOV A, #'B'
        RET
K7:       MOV A, #'7'
        RET
K8:       MOV A, #'8'
        RET
K9:       MOV A, #'9'
        RET
KC:       MOV A, #'C'
        RET
KSTAR:     MOV A, #'*'
        RET
K0:       MOV A, #'0'
        RET
KHASH:     MOV A, #'#'
        RET
KD:       MOV A, #'D'
        RET
DEBOUNCE_20MS:
```

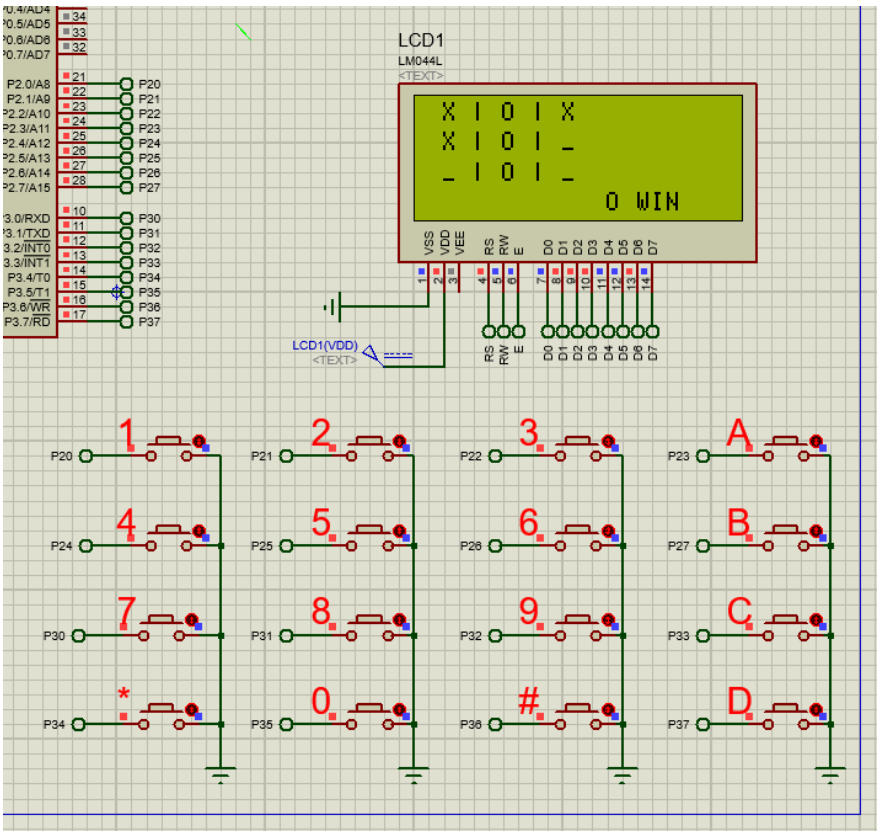
```
        MOV R5, #50
DB1:    MOV R6, #255
DB2:    DJNZ R6, DB2
        DJNZ R5, DB1
        RET
ASCII_TO_CELL:
        MOV TMPA, A
        CLR C
        SUBB A, #'1'
        JC  ATC_BAD
        MOV A, TMPA
        CLR C
        SUBB A, #':'
        JNC ATC_BAD
        MOV A, TMPA
        CLR C
        SUBB A, #'0'
        RET
ATC_BAD:
        MOV A, #0
        RET
        END
```

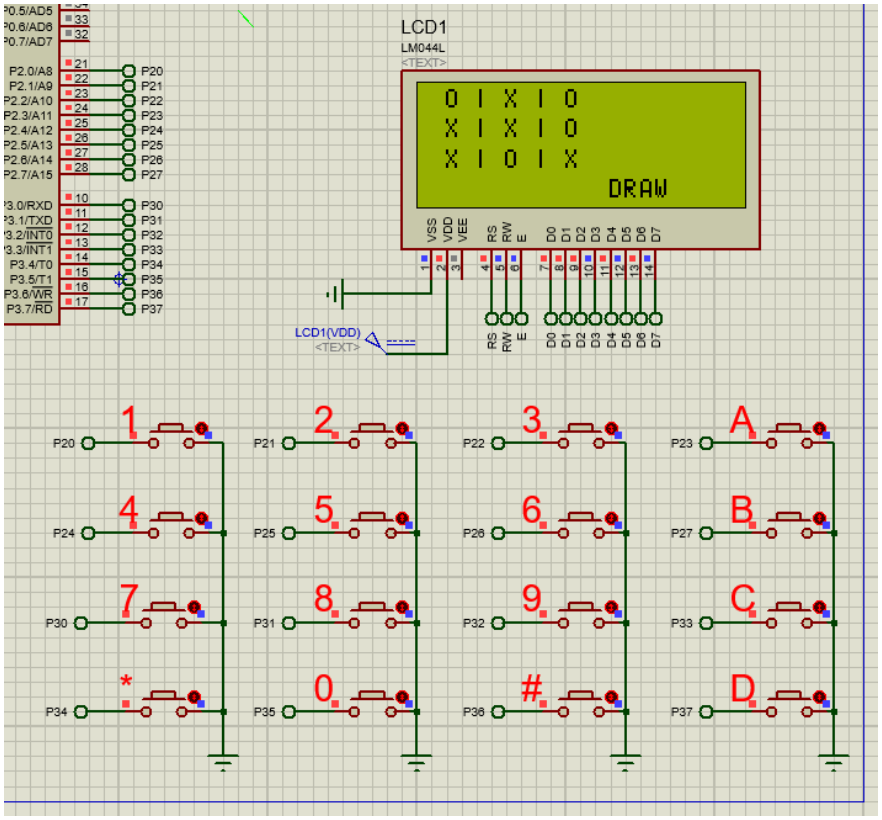
Simulation results:

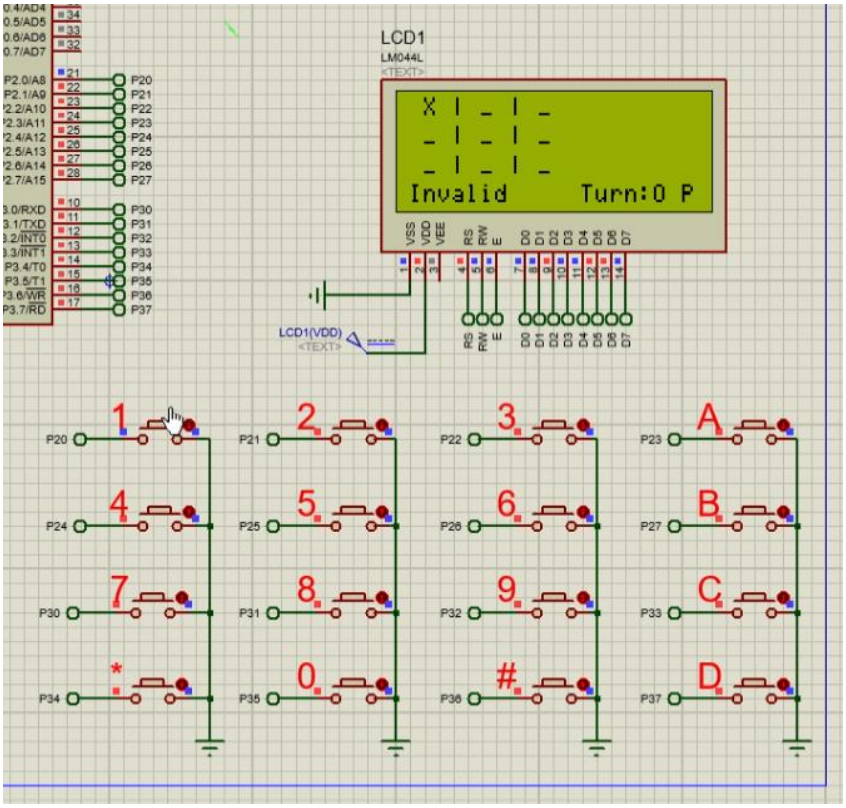



LCD1
LM044L
Turn:X P
LCD1(VDD)
P20
P21
P22
P23
P24
P25
P26
P27
P30
P31
P32
P33
P34
P35
P36
P37
1
2
3
A
4
5
6
B
7
8
9
C
*
0
#
D


LCD1
LM044L
X | O | O
_ | X | _
_ | _ | X
X WIN
LCD1(VDD)
P20
P21
P22
P23
P24
P25
P26
P27
P30
P31
P32
P33
P34
P35
P36
P37
1
2
3
A
4
5
6
B
7
8
9
C
*
0
#
D


LCD1
LM044L
X I O I X
X I O I _
_ I O I _
O WIN
VSS VDD VEE RS RW E D0 D1 D2 D3 D4 D5 D6 D7
LCD1(VDD)
P20 P21 P22 P23 P24 P25 P26 P27 P30 P31 P32 P33 P34 P35 P36 P37
1 2 3 A 4 5 6 B 7 8 9 C * 0 # D


LCD1
LM044L
O I X I O
X I X I O
X I O I X
DRAW
VSS VDD VEE RS RW E D0 D1 D2 D3 D4 D5 D6 D7
LCD1(VDD)
P20 P21 P22 P23 P24 P25 P26 P27 P30 P31 P32 P33 P34 P35 P36 P37
1 2 3 A 4 5 6 B 7 8 9 C * 0 # D


LCD1
LM044L
X | _ | _
_ | _ | _
_ | _ | _
Invalid Turn:0 P
VSS
VDD
VEE
RS
RW
E
D0
D1
D2
D3
D4
D5
D6
D7
LCD1(VDD)
P20
P21
P22
P23
P24
P25
P26
P27
P30
P31
P32
P33
P34
P35
P36
P37
1
2
3
A
4
5
6
B
7
8
9
C
*
0
#
D